

OOPs Concepts in Java

06 October 2026 20:19

Object - Oriented Programming is a paradigm that provides concepts such as inheritance, data binding, polymorphism, etc.

Simula is considered the first object - oriented programming language. The programming paradigm where everything is represented as an object is known as pure Object Oriented Programming Language.

Smalltalk is considered the first truly object - oriented programming language.

The popular object - oriented languages are Java, Javascript, C #, PHP, Python, C++, etc.

OOPs(Object - Oriented Programming)

Object - Oriented Programming aims to implement real - world entities. For example objects, classes, abstraction, inheritance, polymorphism, etc.

Object means the real-world entities such as pen, chair, table, computer, watch, etc. **Object-oriented Programming** is a methodology or paradigm for designing a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- **Object**
- **Class**
- **Inheritance**
- **Polymorphism**
- **Abstraction**
- **Encapsulation**

Apart from these concepts, there are some other terms which are used in Object - Oriented Design:

- **Coupling**
- **Cohesion**
- **Association**
- **Aggregation**
- **Composition**

Object

Any entity has **state** and **behavior** is known as object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

An **Object** can be defined as an **instance** of a class. It contains an address and takes up some space and memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.

Example: A dog is an object because it has states like color, name, breed, cetera as well as behaviors like wagging the tail, barking,

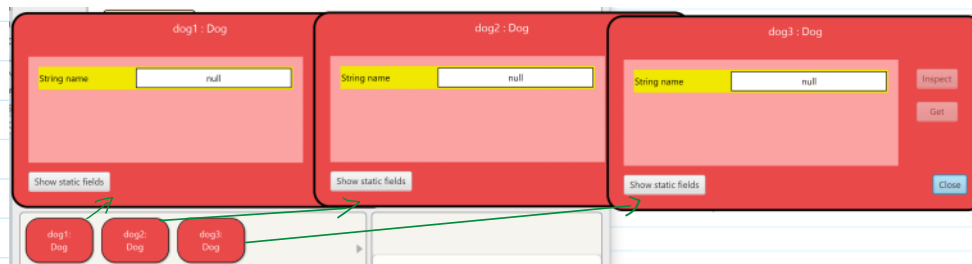
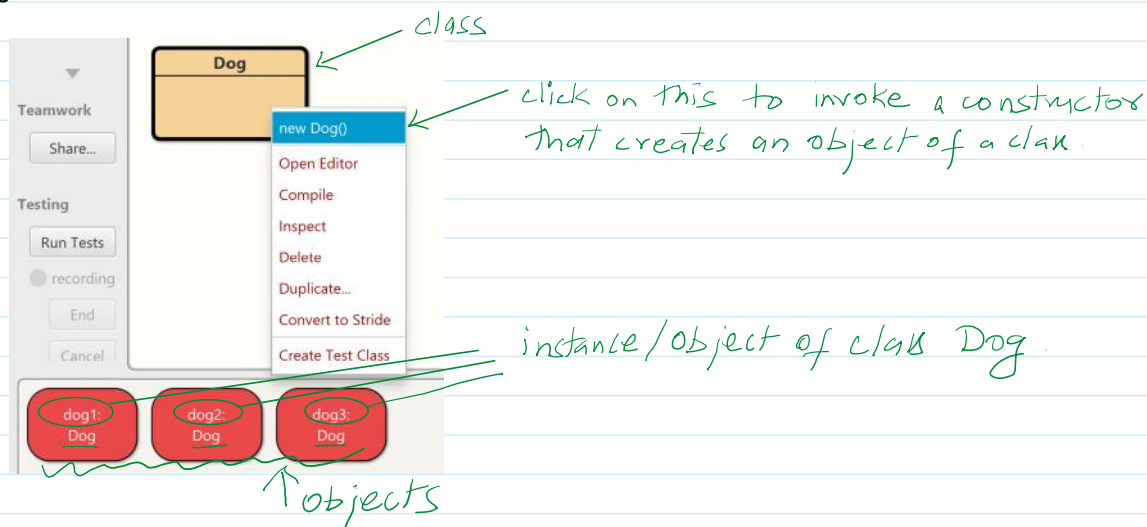
eating, etc.

The primary element of an object are:

- **State:** It is embodied in an object's properties. Additionally it reflects and objects attribute.
- **Behavior:** An object's methods are representation of it. Additionally, it represents how an object reacts to other object.
- **Identity:** Identity is a special name that gives a thing the ability to communicate with other objects.
- **Method:** A method is a group of statements that carry out a certain operation and gives an outcome. A method can complete a certain task without giving back any results. Methods are regarded as time saver since they enable us to reuse the code without having to type it again. Java differs from languages like C, C++ and Python in that each method needs to be a component of a class.

Example of Object:

The following example demonstrates how to create a class. Instance an object, assign values to object fields and invoke method using that object.



Every object has its own attribute called name:

To access these attributes we have to use object-name.variable-name.

identifies object names

we change the state of each object.

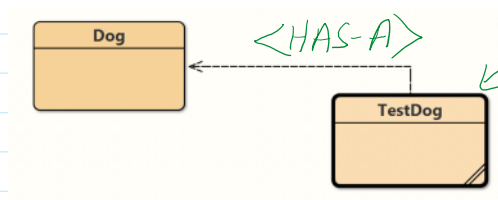
```
dog1.name = "tintin";
dog2.name = "Nemo";
dog3.name = "Polo";
```

Common behavior of each object.

Methods

```
Blue: Terminal Window - Learning_Java
Options
Polo say's Woof!
Nemo say's Woof!
tintin say's Woof!
```

is called using object, the method uses objects attribute(s), data to execute its code.



This class contains the Objects of Dog class.

TestDog —> <HAS-A> Dog

```
package ClassAndObject;
```

```
/**
 * Write a description of class Dog here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Dog {
    String name; //property or attribute or state
```

access modifier

Java keyword

class name

```

{
    String name; //property or attribute or state

    void bark(){ //behavior
        System.out.print("\n"+name + " say's Woof!");
    }
}

```

class name (pointing to the opening curly brace)

Code reused (pointing to the `bark()` method)

```

package ClassAndObject;

/**
 * Write a description of class TestDog here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestDog
{
    public static void main(String[] args){
        System.out.print("\n");
        Dog dog1 = new Dog(); //creating objects
        Dog dog2 = new Dog();
        Dog dog3 = new Dog();
        dog1.name = "tintin"; // Assigning values to the objects field.
        dog2.name = "Nemo";
        dog3.name= "Polo";

        //calling method on the object(s)
        dog1.bark();
        dog2.bark();
        dog3.bark();
    }
}

```

Output:

BlueJ: Terminal Window - Learning_Java

Options

```

tintin say's Woof!
Nemo say's Woof!
Polo say's Woof!

```

Class

Class is a logical entity. A class can also be defined as a blueprint from which we can create an individual object. Class does not consume any space.

In general, a class declaration includes the following in the order as it appears:

1. Modifiers: A class can be public or have default access.

2. **class keyword:** The class keyword is used to create a class.
3. **Class name:** To begin with an initial letter (capitalized by convention)
4. **Superclass(if any):** The name of the class's parent (super class) if any preceded by the keyword **extends**. A class can only extend (subclass) 1 parent.
5. **Interface(if any):** A comma - separated list of interfaces implemented by the class, if any preceded by the keywords **implements**. A class can implement more than one interface.
6. **Body:** The class body surrounded by braces, {}.

Encapsulation

Binding (or wrapping) Code and data together into a single unit is known as **encapsulation**. For example, a capsule is wrapped with different medicines.

A Java class is an example of encapsulation. A Java Bean is a fully encapsulated class because all the data members are private.

